

RAPPORT DE STAGE DE MASTER 1

Année Universitaire 2025 – 2026

Etude numérique de surfaces de potentiel

Présenté par :

Amyne KADDOURI

Formation : Master 1 Calcul Scientifique et
Modélisation

Dates du stage : 18 Mai – 17 Juillet 2026

Encadrement :

Tuteur universitaire : Mariko DUNSEATH-
TERAO

Tuteur d'accueil : Kevin DUNSEATH

Fonction : Chargé de Recherche CNRS

Remerciements

Je tiens tout d'abord à exprimer ma gratitude envers mes responsables de stage, **Mariko DUNSEATH-TERAO** et **Kevin DUNSEATH**, pour leur accueil, leur suivi académique, la qualité de leur encadrement ainsi que de leurs conseils mais aussi pour la confiance qu'ils m'ont accordée tout au long de cette période.

Mes remerciements s'adressent également aux membres de l'équipe du département de physique moléculaire de l'Institut de Physique de Rennes notamment **Franck THIBAUT** pour les échanges, ses conseils et son temps accordé.

Table des matières

Remerciements	1
Table des figures	4
1 Présentation de la Structure d’Accueil	5
2 Fondements et objectifs du stage	6
2.1 Contexte Théorique	6
2.2 Sujet du stage	8
2.3 Le modèle physique et ses paramètres	9
3 Méthodes	10
3.1 Les surfaces étudiées	10
3.2 Les méthodes d’optimisation	10
3.3 L’implémentation du modèle analytique	11
3.3.1 Validation des améliorations	11
3.4 Les critères d’évaluation des ajustements	12
3.5 Approche alternative : Réseaux de neurones	13
4 Résultats	14
4.1 Ajustement de la surface n°1	14
4.2 Ajustement de la surface n°2	16
4.2.1 Le temps de calculs	16
4.2.2 Analyse graphique et quantitative	16
4.3 Nouveau système : Ar-HNC	18
4.4 Réseaux de neurones	19
5 Analyse critique et perspectives	20
5.1 Le choix des méthodes d’optimisation	20
5.2 La qualité des données de référence	20
5.3 Approche par les réseaux de neurones	21
6 Conclusion	22
6.1 Bilan Technique et Scientifique	22
6.2 Bilan Personnel et Professionnel	22
Bibliographie	22
A Annexes	24
A.1 Tableaux	24
A.2 Figures	27

A.3 Code Source	31
---------------------------	----

Table des figures

2.1	Illustration du complexe Ar-HCN	7
2.2	Coupe de la PES à $\theta = 90^\circ$ – Complexe Ar-HCN	7
2.3	Coupes de surface pour R fixé – Comparaison des paramètres de R.R. Toczyłowski aux données <i>ab initio</i>	8
4.1	Coupes de surface pour R fixé – Comparaison des paramètres aux données <i>ab initio</i>	14
4.2	Erreur relative en % pour θ fixé	15
4.3	Coupes de la PES pour les quatres paramètres sélectionnés – Surface n°2	17
4.4	Coupes de la PES pour les surfaces n°2 et n°3	18
4.5	Coupes de la PES pour les surfaces n°2 et n°3	19
A.1	Contour plot de l'énergie potentielle pour Ar-HCN (surface n°1)	27
A.2	Coupe de la PES pour $R = 7.5 a_0$ sur quatre configurations de <code>least_squares</code> avec un même résidu défini par l'erreur absolue – surface n°2	28
A.3	Coupes de surface pour θ fixé – Ar-HCN (surface n°2)	29
A.4	Contour plot de l'énergie potentielle pour Ar-HCN (surface n°2)	30

1. Présentation de la Structure d'Accueil

L'Institut de Physique de Rennes [1] (IPR UMR 6251) est une Unité Mixte de Recherche qui a pour tutelles l'Université de Rennes et le CNRS. Au CNRS, l'IPR est rattaché principalement au CNRS Physique et secondairement au CNRS Chimie et au CNRS Ingénierie. Au sein de 5 bâtiments situés sur le campus de Beaulieu, les 6 départements de recherche mènent des activités scientifiques extrêmement variées et une grande partie des thématiques de la physique y est abordée.

Les travaux menés relèvent essentiellement de la recherche fondamentale sur des concepts émergents de la physique contemporaine et en réponse à des questions sociétales, avec une part de plus en plus importante d'activités qui se développent dans le cadre de partenariats industriels.

L'IPR développe des recherches fondamentale et appliquée au meilleur niveau international avec le soutien des personnels techniques et administratifs. Une des forces majeures de l'IPR réside dans l'imbrication entre projets de recherche et développement de savoir-faire technique et technologique.

Le département "Physique Moléculaire" fédère théoriciens et expérimentateurs autour de la structure et de la dynamique de systèmes atomiques et moléculaires et de leur interaction avec la lumière.

2. Fondements et objectifs du stage

2.1 Contexte Théorique

On considère un système composé d'un atome A et d'une molécule linéaire B de longueur fixe (rotateur rigide). La structure et la dynamique du système sont décrites par la mécanique quantique via une fonction d'onde $\Psi(\vec{R}, \vec{\rho})$:

- $\vec{R}$: Position du centre de masse de A par rapport au centre de masse de B
- $\vec{\rho}$: Coordonnées internes de A et B (électrons, etc.)

solution de l'équation de Schrödinger.

La masse des électrons étant beaucoup plus petite que celle du noyau, ceux-ci s'adaptent presque instantanément aux variations des positions nucléaires. L'approximation de Born-Oppenheimer consiste à séparer la fonction d'onde :

$$\Psi(\vec{R}, \vec{\rho}) = F(\vec{R})\chi(\vec{\rho}; \vec{R})$$

- $F(\vec{R})$: qui décrit le mouvement relatif
- $\chi(\vec{\rho}; \vec{R})$: qui décrit le mouvement électronique.

Cette approche permet de résoudre le problème en trois étapes :

1. On fixe la géométrie nucléaire $\vec{R}$ et on résout l'équation de Schrödinger électronique pour obtenir $\chi(\vec{\rho}; \vec{R})$.
2. En faisant varier $\vec{R}$, on obtient l'énergie électronique en fonction de cette géométrie : c'est la **surface d'énergie potentielle (PES)**.
3. On résout l'équation du mouvement nucléaire pour obtenir $F(\vec{R})$.

La surface d'énergie potentielle est obtenue par des techniques de chimie quantique et des programmes informatiques associés. Les calculs étant particulièrement coûteux, ceux-ci sont effectués sur un ensemble réduit de géométries nucléaires (**valeurs de potentiel *ab initio***). On utilise ensuite un modèle analytique pour interpoler la fonction de potentiel $V(R, \theta)$ car les calculs de dynamique requièrent a priori la valeur du potentiel en n'importe quel point. La précision de l'interpolation est cruciale pour la fiabilité des études de dynamique quantique sur surfaces de potentiel.

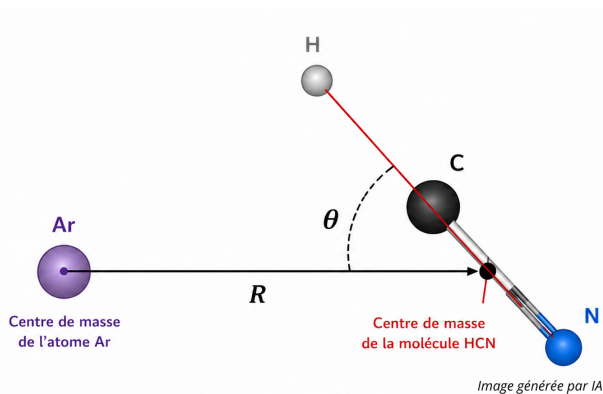


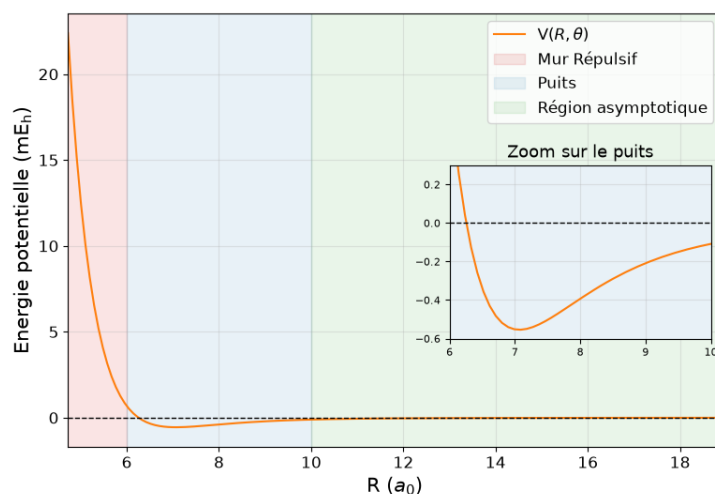
FIGURE 2.1 – Illustration du complexe Ar-HCN

On suppose que la surface de potentiel ne dépend que de R et θ car la molécule est rigide (elle ne vibre pas) et est invariante sous rotation autour de son axe.¹

- R : Distance entre les centres de masse de A et B.
- θ : Angle entre l'axe de la molécule B et le vecteur $\vec{R}$.

En pratique, en fixant une variable (par exemple $\theta = 90^\circ$) on réalise des coupes de la surface d'énergie potentielle. On fixe à zéro l'énergie du complexe correspondant à la limite des très grandes distances ($R \rightarrow \infty$). Le signe de l'énergie indique la nature de l'interaction entre A et B ($V > 0$: Répulsive ; $V < 0$: Attractive).

- À courte distance $V(R, \theta)$ est très grand en raison du potentiel centrifuge (mur répulsif).
- À grande distance $V(R, \theta) \rightarrow 0$ par valeurs négatives (région asymptotique).
- La configuration stable du système se trouve dans le puits de potentiel.

FIGURE 2.2 – Coupe de la PES à $\theta = 90^\circ$ – Complexe Ar-HCN

1. Pour le système étudié ici, le PES correspond à l'état fondamental électronique du complexe Ar-HCN, c'est-à-dire l'état présentant l'énergie électronique la plus faible.

Dans ce travail, nous utilisons le milli-Hartree² (mE_h) comme unité d'énergie, le bohr (a_0) comme unité de distance et le degré comme unité angulaire. On précise que lorsque $\theta = 0^\circ$, on a Ar-HCN linéaire.

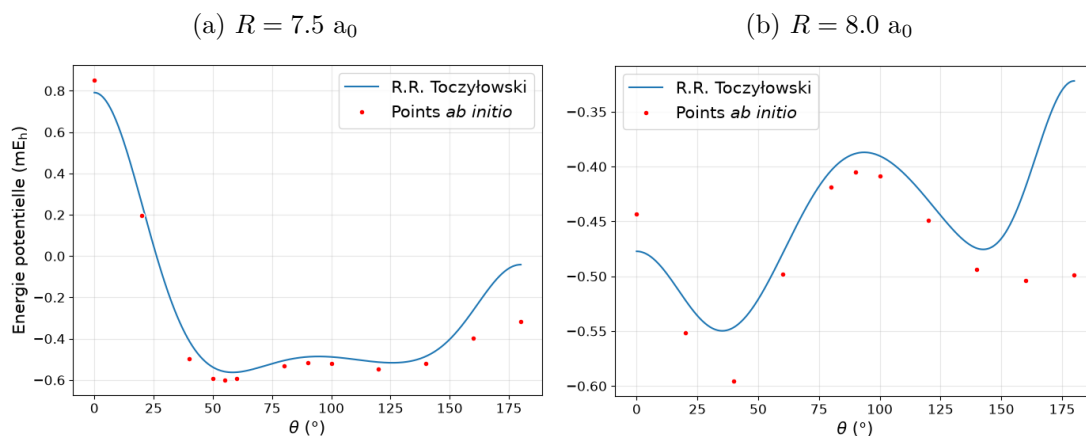
2.2 Sujet du stage

La problématique de ce stage trouve son origine dans l'article de référence publié en 2000 par R.R. Toczyłowski, F. Doloresco et S.M. Cybulski : *Theoretical study of the He-HCN, Ne-HCN, Ar-HCN, and Kr-HCN complexes* [2].

Cet article présente des données *ab initio* pour ces quatre systèmes, ainsi qu'un modèle analytique permettant de calculer la surface d'énergie potentielle $V(R, \theta)$. Ce modèle repose sur 40 paramètres qui doivent être ajustés en fonction du système étudié. Toutefois, les auteurs ne mentionnent pas les valeurs numériques de ces paramètres.

Quatre ans plus tard, la thèse de doctorat de R.R. Toczyłowski (2004) propose un jeu de paramètres pour chacun des systèmes étudiés. Cependant, une incohérence majeure est apparue : l'utilisation des paramètres fournis pour l'Argon sur le modèle analytique ne permet pas de reproduire correctement la PES et conduit à des calculs d'états liés insatisfaisants, qui ne correspondent pas aux résultats présentés dans l'article initial.

FIGURE 2.3 – Coupes de surface pour R fixé – Comparaison des paramètres de R.R. Toczyłowski aux données *ab initio*



L'objectif de ce travail est d'une part de réaliser de nouveaux ajustements de la surface (fits) d'énergie potentielle afin de déterminer des jeux de paramètres qui conduisent aux valeurs du potentiel *ab initio* et permettent de reproduire les énergies d'états liés du système, et d'autre part, de définir des critères pertinents pour évaluer et quantifier la qualité de ces ajustements.

2. 1 Hartree = 4.36×10^{-18} joules

2.3 Le modèle physique et ses paramètres

Le modèle décrivant $V(R, \theta)$ s'exprime comme la somme d'un terme qui décrit les interactions à courte (*short*) portée (V_{sh}) et d'un terme à très longue portée (V_{as}) dans la région asymptotique. Entre les deux se trouve la région du puits où l'atome et la molécule peuvent se trouver piégés temporairement et interagir de manière significative durant une collision.

$$V(R, \theta) = V_{sh}(R, \theta) + V_{as}(R, \theta)$$

$$V_{sh} = G(R, \theta)e^{D(\theta)+B(\theta)R}$$

avec

$$G(R, \theta) = \sum_{l=0}^5 (g_{0,l} + g_{1,l}R + g_{2,l}R^2 + g_{3,l}R^3)P_l(\cos \theta)$$

$$D(\theta) = \sum_{l=0}^5 d_l P_l(\cos \theta) \quad B(\theta) = \sum_{l=0}^5 b_l P_l(\cos \theta)$$

$P_l(\cos \theta)$ est une forme de base des fonctions en θ et l correspond au moment angulaire (entier, positif ou nul).

$$V_{as} = f_6(|B(\theta)R|) \frac{c_0 P_0(\cos \theta) + c_2 P_2(\cos \theta)}{R^6} + f_7(|B(\theta)R|) \frac{c_1 P_1(\cos \theta) + c_3 P_3(\cos \theta)}{R^7}$$

La fonction d'amortissement $f_n(x)$ vient corriger le comportement non physique à courte distance du développement en $1/R^n$.

$$f_n(x) = 1 - e^{-x} \sum_{k=0}^n \frac{x^k}{k!}$$

Les quarante paramètres à ajuster selon le système étudié sont :

- $b_l, d_l, g_{i,l}$ ($l = 0, \dots, 5$ et $i = 0, 1, 2, 3$).
- c_0, c_1, c_2, c_3 .

3. Méthodes

3.1 Les surfaces étudiées

- **Surface n°1 – Ar-HCN** : Cette première surface publiée dans l'article de référence [2] contient 140 points *ab initio* sur une grille de R et θ incomplète (Table A.1). R (resp. θ) prend ses valeurs entre 5 et 15 a_0 (resp. 0° et 180°) avec un pas irrégulier.
- **Surface n°2 – Ar-HCN** : Publiée dans le *Chinese Journal of Chemical Physics* [3], elle contient 1767 points *ab initio* pour une grille régulière de 93 valeurs de R (de 3.78 à 47.24 a_0) et 19 valeurs de θ (de 0° à 180°).
- **Surface n°3 – Ar-HNC** : Cette troisième surface [5] contient une combinaison de 25000 points *ab initio* et issus d'un premier fit sur une grille régulière de 500 valeurs de R (entre 4 et 30 a_0) et 50 valeurs de θ (entre 0° et 180°).

3.2 Les méthodes d'optimisation

Pour effectuer nos ajustements de surface, on utilisera le langage Python et les fonctions du module `scipy.optimize` : `curve_fit` [6], `least_squares` [7], `differential_evolution` [8].

- `curve_fit` : Cette méthode est la plus directe pour ajuster une fonction modèle $y = f(x, p)$ ($x = (R, \theta)$ dans notre cas). Elle repose sur des algorithmes des moindres carrés non linéaires comme Levenberg-Marquardt ou Trust Region Reflective.
En pratique, il suffit d'y entrer en argument notre fonction modèle, les données de référence ainsi que des paramètres initiaux p_0 . La fonction convergera vers les paramètres optimaux les plus proches de p_0 . Pour améliorer les résultats, on utilisera les options permettant de pondérer ou non le fit et faisant varier la méthode de minimisation utilisée par la fonction.
- `least_squares` : Plus générale que `curve_fit`, cette méthode travaille sur une fonction résidus fournie par l'utilisateur ($\text{Res}(p) = y_{\text{ref}} - y_{\text{pred}}(p)$ par exemple). Elle offre un niveau de contrôle plus poussé avec notamment la possibilité de fixer des bornes sur les paramètres à calculer ou faire varier les fonctions de perte utilisées. La fonction convergera, à l'instar de `curve_fit`, vers le minimum local le plus proche de p_0 .
- `differential_evolution` : Contrairement aux deux méthodes précédentes qui sont des optimisateurs locaux dépendants d'un bon choix du point initial p_0 , `differential_evolution` est un algorithme d'optimisation globale stochastique. Il permet d'explorer un vaste espace de valeurs à l'aide d'une population de solutions p candidates, d'isoler la région du minimum global pour ensuite effectuer un affinage local.
Comme `least_squares`, elle ne prend pas directement le modèle en argument mais une fonction de coût que l'on choisit.

3.3 L'implémentation du modèle analytique

Dans un premier temps, l'objectif est de développer une fonction Python fidèle au modèle de V (Voir Section 2.3). Il est bon de préciser que dans notre programme pour la partie à courte portée, on prendra $e^{D(\theta)-B(\theta)R}$ au lieu de $e^{D(\theta)+B(\theta)R}$ car c'est ce que l'on retrouve plus communément dans la littérature¹.

Après avoir vérifié la validité des résultats fournis par la fonction, il est essentiel d'en optimiser les performances d'exécution. En effet, les algorithmes d'optimisation évaluant le potentiel plusieurs milliers de fois, de légères réductions du temps de calcul auront un impact majeur sur la durée globale du fit. Pour y parvenir, plusieurs axes d'amélioration ont été examinés :

- **Vectorisation NumPy** : Remplacement des boucles Python et du module `math` par des opérations vectorielles, permettant d'évaluer simultanément l'ensemble des points (R, θ) .
- **Optimisations algébriques** : Utilisation du schéma de Horner pour les polynômes $G(R, \theta)$ et f_n , pré-calcul explicite des factorielles, et évaluation des polynômes de Legendre avec `np.polynomial.legendre.legval`.
- **Stabilité numérique** : Utilisation de `np.clip` et traitement des NaN ou infinis (`np.nan_to_num`), pour prévenir les problèmes d'overflow lorsque les paramètres testés par les algorithmes d'optimisation sont absurdes.

(Les deux versions l'implémentation sont présentées en annexe : A.1 – A.2.)

3.3.1 Validation des améliorations

Pour s'assurer de la pertinence des modifications effectués on applique une série de tests.

Dans un premier temps, il s'agit de vérifier que les deux fonctions sont numériquement équivalentes. Pour ce faire, on tire aléatoirement 500 valeurs de R (resp. θ) entre 2 et $10 a_0$ (resp. 0° et 180°) selon une loi uniforme² et notre jeu de 40 paramètres selon une loi normale centrée réduite $\mathcal{N}(0, 1)$ ³. On compare ensuite la différence des résultats obtenus avec les deux fonctions et on s'assure qu'elle soit suffisamment faible.

TABLE 3.1 – Comparaison des sorties de V_{init} et V_{opt} ($|V_{init} - V_{opt}|$)

Différence maximale	Différence moyenne	Écart-type de la différence
1.788×10^{-6}	1.220×10^{-8}	1.090×10^{-7}

1. On s'assurera d'ajuster les signes des c_l lorsque nécessaire

2. `numpy.random.uniform(2, 10, 500)`

3. `numpy.random.randn(40)`

Dans un second temps, on évalue les gains en performance par trois tests :

- **Sur la fonction isolée** : Mesure du temps moyen d'exécution de V sur 10000 itérations avec `perf_counter`.
- **Sur les fonctions résidus** : Mesure du temps minimal sur 5 cycles de 10000 évaluations avec `timeit.repeat()` (pour filtrer de possibles interférences de processus indépendants qui tourneraient en arrière plan).
- **Sur la résolution complète (least_squares)** : Mesure du temps moyen de fit sur 1000 p_0 tirés aléatoirement, pour évaluer le gain en conditions réelles.

Pour éviter les biais de mesure, on réalise les tests dans des conditions expérimentales identiques par l'intégration de *warm-up*⁴ et le nettoyage de la mémoire.

TABLE 3.2 – Temps d'exécution moyen par appel en secondes

	V_{init}	V_{opt}	Gain
Isolée	3.044×10^{-3}	8.334×10^{-4}	$\times 3.65$
Résidus	2.495×10^{-4}	1.117×10^{-4}	$\times 2.23$
least_squares	11.336	4.942	$\times 2.29$

3.4 Les critères d'évaluation des ajustements

Après avoir déterminé des paramètres optimaux à l'aide des différentes méthodes, il est essentiel d'évaluer la qualité de ces ajustements. Pour s'assurer que le modèle incluant les nouveaux paramètres reproduise bien les données, on effectue une première analyse graphique que l'on précisera à l'aide de métriques quantitatives.

La vérification graphique est la première étape qui permet d'écarter les résultats aberrants et d'identifier de possibles faiblesses dans les méthodes utilisées.

- **Superposition aux données** : Comparaison directe des courbes obtenues aux données *ab initio* à l'aide de coupes de la surface sur plusieurs R ou θ fixés
- **Cartes d'erreurs** : Représentation graphique des erreurs en fonction de la géométrie

Ensuite, pour mesurer de manière plus précise la qualité, on introduit trois métriques : R^2 , RMSE et MAE. On note V les données de référence, $\bar{V}$ la moyenne de ces données et $\hat{V}$ les valeurs obtenues avec le modèle analytique.

- **Coefficient de détermination (R^2)** : Indicateur sans dimension qui informe sur la qualité globale du fit. On vise à obtenir $R^2 = 1$.

$$R^2 = 1 - \frac{\sum_{i=1}^N (V_i - \hat{V}_i)^2}{\sum_{i=1}^N (V_i - \bar{V})^2}$$

4. Exécution préliminaire que l'on ne comptabilise pas. Elle permet de précharger la fonction et les données dans le cache du CPU

- **Erreur quadratique moyenne (RMSE)** : Pénalise les grands écarts locaux. Elle prend la même unité que les données que l'on cherche à évaluer.

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (V_i - \hat{V}_i)^2}$$

- **Erreur absolue moyenne (MAE)** : Accorde un poids linéaire à toutes les erreurs. Elle prend la même unité que les données étudiées.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |V_i - \hat{V}_i|$$

3.5 Approche alternative : Réseaux de neurones

Durant la dernière semaine du stage, nous avons tenté d'apporter une alternative à l'ajustement du modèle analytique. On pose un modèle semi analytique :

$$V(R, \theta) = \sum_{l=0}^{L_{\max}} c_l(R) \cdot P_l(\cos \theta)$$

On cherche à implémenter avec le module PyTorch un réseau de neurones qui apprendra les fonctions représentant les coefficients $c_l(R)$. Pour ce faire, par souci de temps, j'ai d'abord présenté le problème à Gemini pour obtenir un premier prototype à ajuster si nécessaire. L'architecture de la procédure est composée de trois étapes :

- **Réseau dense** : Un réseau de neurones prend la distance R (que l'on normalise) en entrée et prédit le vecteur des $L_{\max} + 1$ coefficients $c_l(R)$. On prend comme fonction d'activation : GELU (Gaussian Error Linear Unit).
- **Polynômes de Legendre** : Les $P_l(\cos \theta)$ sont évalués en parallèle.
- **Fusion** : L'énergie totale est le produit scalaire entre le vecteur des coefficients $c_l(R)$ et le vecteur des polynômes $P_l(\cos \theta)$.

On fixe $L_{\max} = 18$ et on effectue l'entraînement sur la surface n°3 (3.1). Pour identifier la configuration optimale, on fait varier cinq hyperparamètres :

- **batch_size** : taille du lot.
- **hidden_layers** : nombre et dimensions des couches cachées.
- **learning_rate** : taux d'apprentissage.
- **loss** : fonction que l'on cherche à minimiser (MAE ou Erreur relative A.3).

On fixe le nombre d'époques d'entraînement à 1000 ou 2000 et on sélectionne le meilleur checkpoint.

4. Résultats

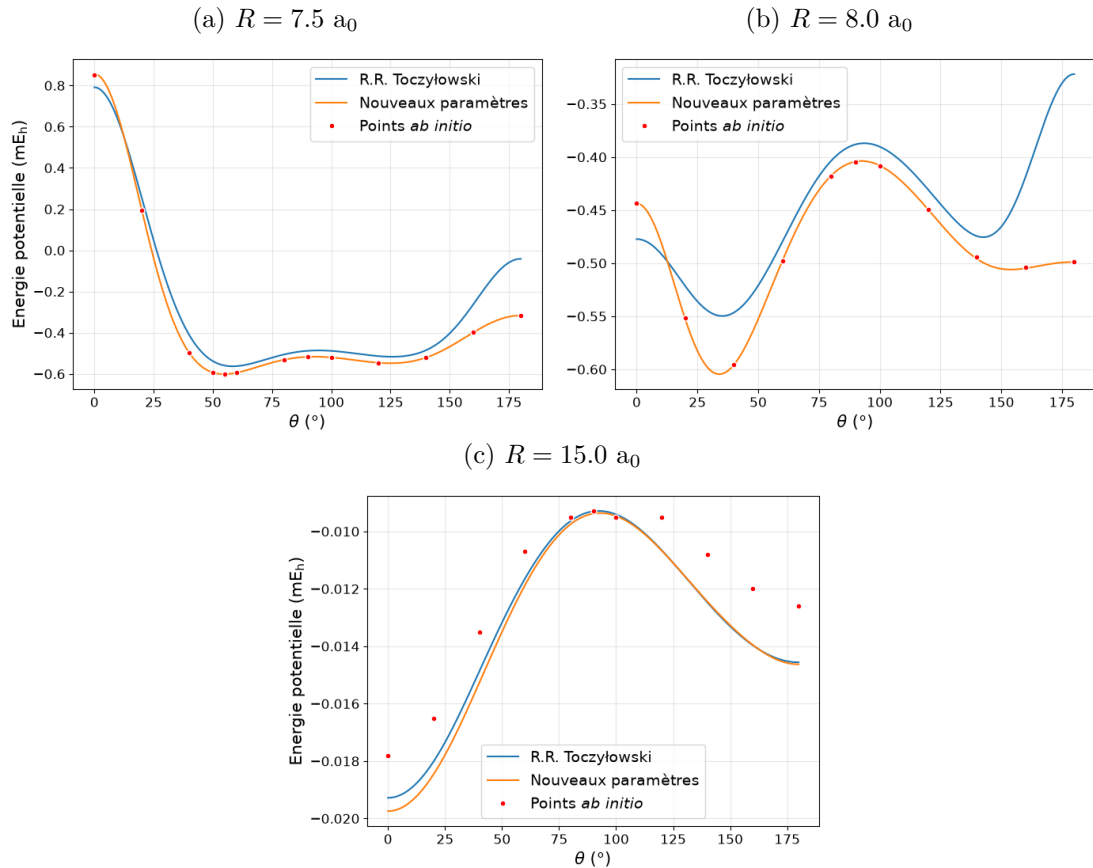
4.1 Ajustement de la surface n°1

Pour l'ajustement de cette première surface d'énergie potentielle [2], seules `curve_fit` et `least_squares` ont été employées. On se satisfait d'une fonction de résidus standard (définie par $|V_{\text{ref}} - V_{\text{pred}}|$) pour la résolution via `least_squares`.

Le processus d'ajustement n'a présenté aucune difficulté particulière. Les résultats se sont avérés relativement stables face aux variations des hyperparamètres des méthodes : les modèles obtenus avec les nouveaux paramètres sont très similaires sur le plan graphique et quantitatif (voir Tableau A.4).

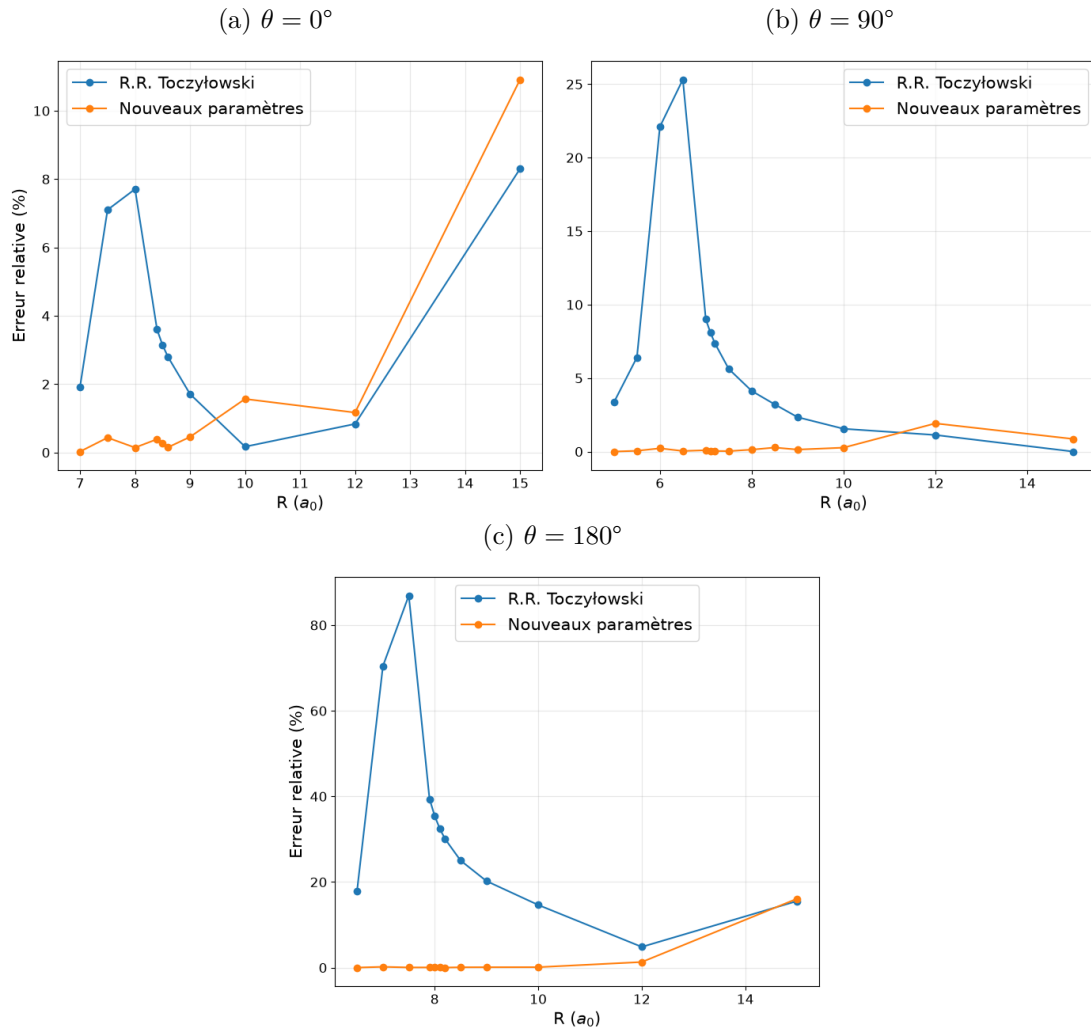
On choisit de retenir pour cette surface le jeu de paramètres obtenu avec `least_squares`, configuré avec la mise à l'échelle `x_scale='jac'`¹ et la fonction de perte `loss='huber'`².

FIGURE 4.1 – Coupes de surface pour R fixé – Comparaison des paramètres aux données *ab initio*



1. Met à l'échelle les paramètres en utilisant le Jacobien de la fonction résidu.
2. Réduit l'influence des valeurs aberrantes lors de l'optimisation.

Le gain en qualité des nouveaux paramètres sur ceux donnés par R.R. Toczyłowski [4] ne fait pas de doute. Toutefois, en examinant certaines coupes à grande distance (voir Figure 4.1c), on observe une difficulté du modèle à garder une erreur relative constante. En effet, dans la région asymptotique, l'énergie d'interaction tend vers zéro. De ce fait, un écart minime est beaucoup plus marqué.

FIGURE 4.2 – Erreur relative en % pour θ fixé

Des figures complémentaires appuyant les observations sont présentées dans l'Annexe (A.1).

4.2 Ajustement de la surface n°2

L'ajustement de cette deuxième surface s'est avéré plus complexe que le précédent. Ceci est dû à un volume de données plus important et à un intervalle d'étude plus large pour la distance R ³. On utilise les méthodes `curve_fit`, `least_squares` et `differential_evolution`.

Afin d'optimiser le fit, plusieurs formulations de l'erreur ont été testées. L'objectif de cela est de donner plus de poids aux valeurs d'énergie dans les zones physiquement importantes du puits et à grande distance, plus faibles que celles de la barrière centrifuge où la fonction d'onde ne pénètre pas. On prend également la décision de réduire l'ensemble d'étude à des valeurs de $R \in [4.7, 18.9] a_0$.

- Pour `least_squares` : On effectue les calculs sur trois fonctions résidus : l'erreur absolue et deux variantes de l'erreur relative
- Pour `differential_evolution` : On effectue les calculs sur cinq fonctions de coûts : l'erreur absolue simple et des variantes pondérées

Lors de l'ajustement de cette surface, les résultats obtenus présentaient beaucoup plus de variations d'une configuration à l'autre.

4.2.1 Le temps de calculs

Les différentes méthodes ne sont pas égales dans le temps nécessaire avant convergence.

TABLE 4.1 – Temps moyen d'exécution des fonctions avant convergence sur la surface n°2

Méthode	Temps moyen en secondes
<code>curve_fit</code>	2.91
<code>least_squares</code>	17.72
<code>differential_evolution</code>	2600.98

`differential_evolution` se distingue par son temps d'exécution considérable.

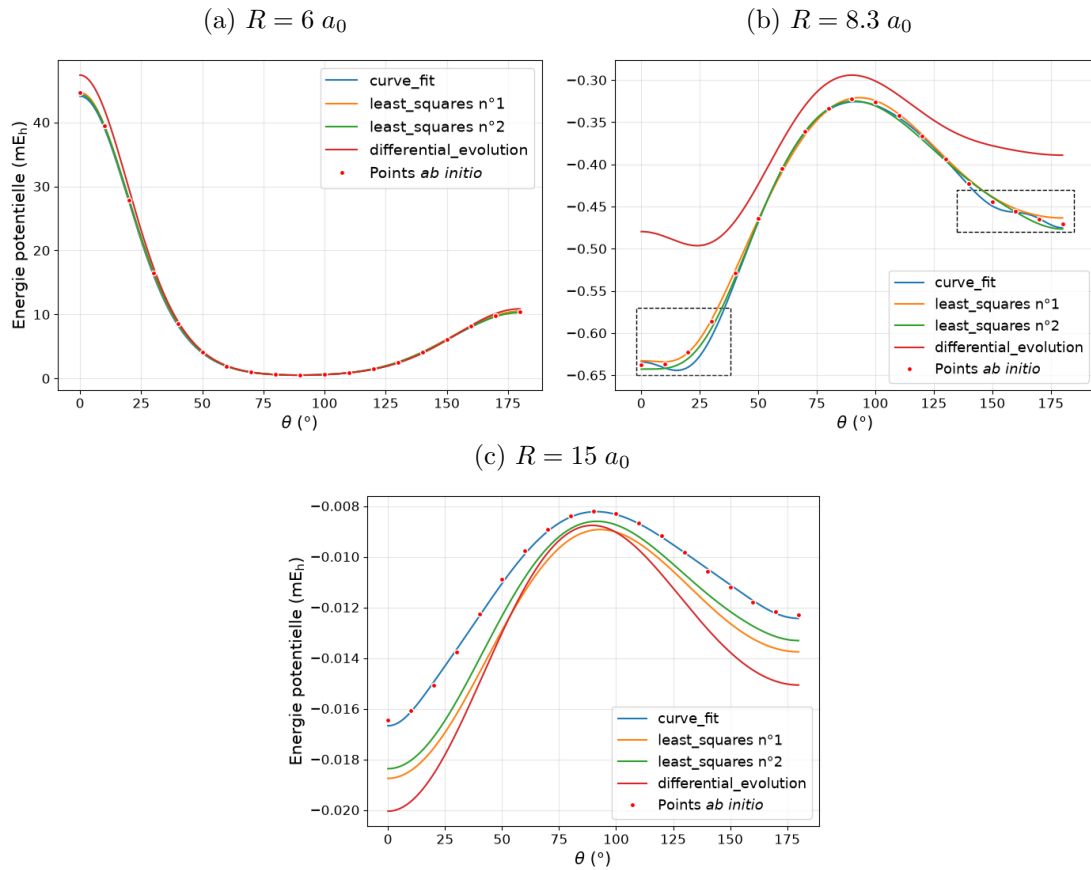
4.2.2 Analyse graphique et quantitative

On effectue une première sélection graphique des paramètres obtenus par les différentes configurations. Par exemple, il apparaît immédiatement que la fonction `least_squares` accompagnée d'un résidu simple (erreur absolue) ne suffit pas (Voir Figure A.2).

Finalement, sur 26 essais, on en sélectionne 4 par le biais des observations graphiques et des métriques.

3. Ce qui implique que l'énergie potentielle prend à la fois des valeurs très grandes et des valeurs qui tendent vers zéro

FIGURE 4.3 – Coupes de la PES pour les quatre paramètres sélectionnés – Surface n°2



On écarte les paramètres obtenus via `differential_evolution` dont les résultats sont visiblement peu satisfaisants. Les autres paramètres donnent des résultats similaires avec chacun ses caractéristiques. Par exemple, les paramètres issus de `curve_fit` sont particulièrement bons pour R grand (Figure 4.3c). Néanmoins, ils présentent une certaine faiblesse aux extrémités pour certaines valeurs de R (voir Figure 4.3b).

On se sert des métriques pour faire un choix et on introduit deux variantes de la MAE : une version pondérée qui donne plus d'importance au puits de potentiel et une version réduite qui est seulement calculée sur l'intervalle $[-1, 1] mE_h$.

TABLE 4.2 – Scores relatifs aux paramètres sélectionnés pour la surface n°2

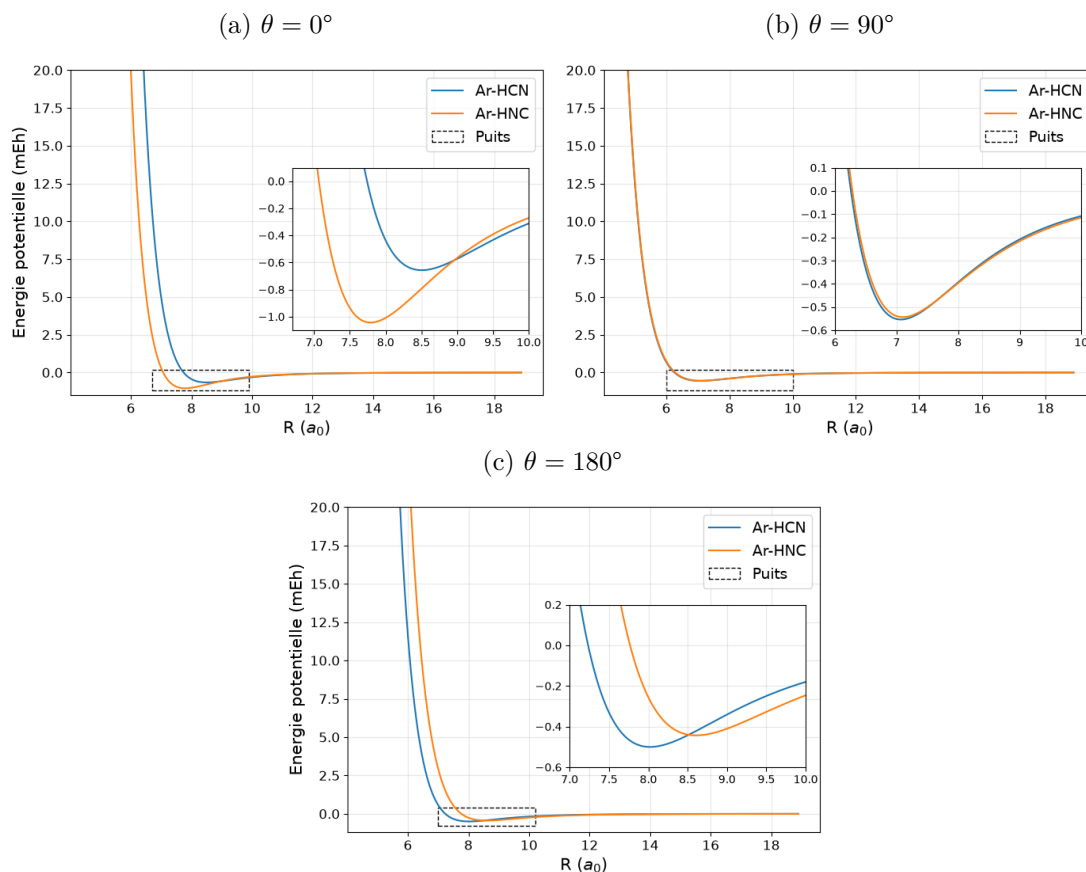
Méthode	R^2	RMSE (mE_h)	MAE (mE_h)	MAE pondérée	MAE intervalle
curve_fit	0.99740565	2.07509219	0.26710798	0.00684283	0.00200393
least_squares n°1	0.99999794	0.05845209	0.01465815	0.00356479	0.00323640
least_squares n°2	0.98918260	4.23725241	0.42103226	0.00664192	0.00237813
differential_evolution	0.98121175	5.58426445	0.87852188	0.03881056	0.02333764

On finit par choisir les paramètres obtenus avec `curve_fit` (Tableau A.5) car bien qu'ils ne se démarquent pas sur le plan quantitatif, lorsque l'on s'intéresse aux erreurs relatives aux alentours des énergies minimales, il apparaît comme meilleur (voir Figure A.4).

4.3 Nouveau système : Ar-HNC

J'effectue un dernier ajustement "classique" sur la surface n°3 (Section 3.1). Les difficultés rencontrées sont intrinsèquement les mêmes que pour la surface n°2. On réitère donc la méthodologie de la Section 4.2.2 et on compare les deux surfaces. (Paramètres appliqués au modèle : Table A.6)

FIGURE 4.4 – Coupes de la PES pour les surfaces n°2 et n°3

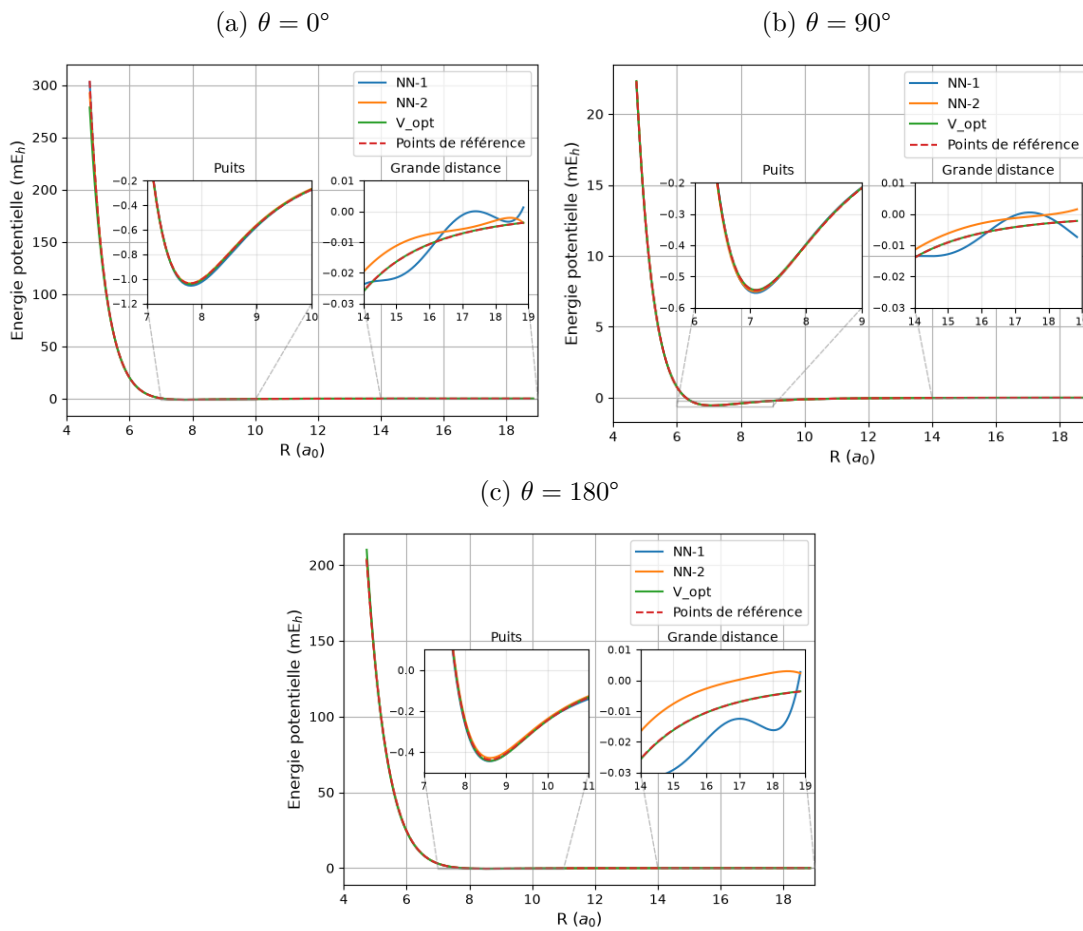


Ces figures mettent en évidence le comportement géométrique lié à l'inversion des atomes de carbone et d'azote. Lorsque l'argon s'approche par les extrémités de la molécule (à 0° et 180°), il fait face à une configuration différente d'un système à l'autre. Ainsi il y a un phénomène d'inversion des puits de potentiel qui apparaît. En revanche, à 90° le centre de la molécule est perçu de la même manière dans les deux cas, d'où la superposition des courbes.

4.4 Réseaux de neurones

L'apprentissage du réseau s'est effectué sur la surface n°3 spécifiquement en raison du grand volume de données qu'il contient pour pouvoir entraîner notre modèle dans les meilleures conditions possible. Sur le plan technique, l'implémentation sur PyTorch a permis de profiter de l'accélération GPU⁴. Ainsi, cette méthode a pu rester très compétitive (temps d'entraînement) par rapport aux algorithmes précédents.

FIGURE 4.5 – Coupes de la PES pour les surfaces n°2 et n°3



Bien que le réseau modélise correctement la région du puits, les figures ci-dessus mettent en évidence ses limites à grande distance, à savoir, des phénomènes d'oscillation et l'apparition de valeurs d'énergie positives. Ceci est dû au fait que le réseau n'est pas informé de la physique du système contrairement au modèle analytique.

4. CUDA 12. – Nvidia GeForce GTX 750ti

5. Analyse critique et perspectives

5.1 Le choix des méthodes d'optimisation

La comparaison des trois algorithmes (3.2) a mis en évidence le fait que `differential_evolution` a produit les résultats les moins satisfaisants graphiquement et quantitativement, et ce, malgré sa capacité théorique à mieux échapper aux minima locaux en plus de présenter des coûts de calculs largement moins intéressants que les deux autres méthodes : 2601 secondes contre 17.7 secondes pour `least_squares`.

L'inefficacité de `differential_evolution` pourrait s'expliquer en premier lieu par une mauvaise configuration des hyperparamètres de la fonction et/ou de l'espace de recherche des paramètres¹, mais également par le fait que les surfaces étudiées étaient probablement plus favorables à l'utilisation de méthodes locales rendant un choix réfléchi de paramètres initiaux² nettement plus intéressant.

5.2 La qualité des données de référence

Le fait de faire varier la nature des données d'entrée a permis de mettre en évidence l'influence de leurs caractéristiques sur la qualité des ajustements.

Par exemple, la surface n°1 (Section 3.1) étant composée d'un faible nombre de points rend fragile la fiabilité des paramètres en dehors des zones denses. Cette fragilité reste anecdotique, du fait que les points sont répartis de manière à viser les régions les plus pertinentes de la surface.

Pour la surface n°3 (Section 3.1) le caractère hybride du jeu de données (points *ab initio* et issus d'un précédent ajustement via un autre modèle) est potentiellement source d'introduction d'un biais dans l'évaluation de qualité des paramètres³. La démarche reste justifiée en raison de l'objectif premier de l'étude de cette surface qui était simplement de fournir des paramètres capables de fitter les données *ab initio*.

Enfin, les surfaces 2 et 3 incluent des distances R extrêmes dont l'intérêt physique est négligeable⁴. Les différences extrêmes d'échelle de l'énergie que cela engendre ajoutent une complexité qui peut facilement être évitée en réduisant l'échantillon aux distances physiquement pertinentes.

1. S'assurer d'un choix de bornes pertinent

2. Comme les paramètres donnés par R.R. Toczyłowski A.2

3. On effectue l'ajustement d'une surface déjà ajustée

4. Le comportement à très courte et très longue distance est connu et ne nécessite pas de détails

5.3 Approche par les réseaux de neurones

Cette étape confirme une limite propre aux modèles basés purement sur les données. En effet, bien que le réseau reproduise correctement la région du puits de potentiel, il tend à diverger rapidement à grande distance (présentant des puits locaux non physiques et des valeurs d'énergie positives). L'absence de contraintes permettant de respecter les comportements physiques du système pénalise cette approche et valorise l'utilisation plus traditionnelle d'un modèle analytique.

Ce constat reste à nuancer car cette approche n'a peut-être pas eu l'occasion de pleinement faire ses preuves, et l'optimisation du réseau aurait pu être poussée davantage notamment via l'introduction d'autres fonctions de perte plus adaptées ou l'utilisation d'autres combinaisons d'hyperparamètres. Il convient également de noter que l'exploration de ces configurations s'est faite sur une carte graphique relativement ancienne⁵, ce qui a potentiellement été un facteur limitant dans l'ampleur de la recherche d'hyperparamètres.

D'autres pistes d'amélioration pourraient éventuellement donner une chance à cette méthode :

- **Intégration de contraintes physiques** : En pénalisant directement les comportements non physiques (*Réseaux neuronaux informés par la physique*).
- **Nouvelle architecture** : En ayant recours à un réseau appliquant une correction pour compenser les erreurs du modèle analytique.
- **Système à dimension supérieure** : Là où les réseaux de neurones gagneraient en pertinence et en compétitivité serait sur des problèmes impliquant davantage de degrés de liberté, par exemple en intégrant les vibrations intramoléculaires au système d'étude.

5. Nvidia GeForce GTX 750ti 2Go : carte sortie en 2014 – Particulièrement modeste au regards des standards actuels

6. Conclusion

6.1 Bilan Technique et Scientifique

Ce stage a tout d'abord été marqué par mon implémentation du modèle analytique permettant de modéliser les surface d'énergie potentielle pour les systèmes composés d'un atome et d'une molécule linéaire. Cette implémentation m'a également demandé un important travail d'optimisation pour rendre viables les futurs ajustements de surface. Ensuite, ce programme a pu être utilisé comme base dans la détermination de nouveaux paramètres validés par nos tests et par calculs d'états liés, permettant de résoudre les erreurs repérées dans les travaux de référence [2]. J'ai également effectué un travail de comparaison des méthodes d'optimisation (comme `curve_fit`, `least_squares` et `differential_evolution`) mettant la lumière sur l'importance du choix des algorithmes en fonction des données d'étude. Enfin, le projet a été vecteur d'une première approche personnelle sur les réseaux de neurones, leur potentiel ainsi que leurs limites.

6.2 Bilan Personnel et Professionnel

Sur le plan technique, ce projet a été source de développement et d'approfondissement de mes compétences en Python pour le calcul scientifique (avec les bibliothèques NumPy, SciPy et PyTorch) mais aussi pour la visualisation de données complexes (notamment via Matplotlib et Plotly) afin de faciliter l'interprétabilité des résultats. J'ai également pu solidifier mes pratiques de développement en travaillant sur l'environnement Linux et par l'utilisation d'outils de gestion de versions (Git). Il était nécessaire au quotidien de faire preuve d'organisation, de rigueur et d'autonomie pour structurer l'évolution du code et gérer les erreurs d'implémentation rencontrées. Enfin, la rédaction de résumés intermédiaires au cours du stage ainsi que la préparation de la soutenance et de ce rapport ont été une occasion de travailler ma communication scientifique.

L'ensemble de mon travail est accessible sur la page GitHub : <https://github.com/Amynek/Stage-M1>.

Bibliographie

- [1] Site officiel de l’Institut de Physique de Rennes : <https://ipr.univ-rennes.fr/linstitut/presentation-de-lipr>
- [2] Toczyłowski R R, Doloresco F and Cybulski S M 2001 J. Chem. Phys. 114 851–864
- [3] T. Liman, S. Tong, N. Chuangang, C. Xialin – 2026 – *Ab initio Potential Energy Surfaces and Low-Temperature Collisional Dynamics for Rotational De-excitation of HNC and HCN by Ar* – Chinese Journal of Chemical Physics, sous presse.
- [4] R.R. Toczyłowski – 2004 – *Doctoral thesis* – Miami University.
- [5] Robin et Lique, communication privée.
- [6] Documentation officielle Scipy https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html.
- [7] Documentation officielle Scipy https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.least_squares.html.
- [8] Documentation officielle Scipy https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html.

A. Annexes

A.1 Tableaux

TABLE A.1 – Énergies d'interaction (en μE_h) calculées pour Ar-HCN [2]

$R[a_0]$	$\theta = 0^\circ$	20°	40°	60°	80°	90°	100°	120°	140°	160°	180°
5.00					13296.3	12669.6	13566.2				
5.50				8192.8	4201.3	3923.2	4239.0	7091.8			
6.00			9684.6	2247.9	780.0	684.1	787.6	1807.4	4728.6	9283.5	
6.50		10758.0	2751.7	98.4	-333.3	-351.9	-323.4	-17.2	987.7	2639.5	3561.2
7.00	5216.5	2908.0	240.4	-524.3	-576.3	-567.9	-564.2	-512.8	-235.5	293.2	601.2
7.50	851.7	195.1	-498.3	-593.8	-531.2	-516.9	-519.4	-546.2	-519.5	-397.9	-316.6
8.00	-443.1	-551.4	-595.6	-498.1	-418.3	-404.8	-408.2	-449.2	-494.0	-503.9	-498.6
8.50	-668.8	-625.9	-502.9	-378.4	-309.4	-299.1	-301.9	-338.8	-392.9	-434.8	-448.9
9.00	-583.2	-515.2	-380.7	-275.6	-223.2	-215.9	-218.7	-247.0	-292.8	-333.8	-349.6
10.00	-315.5	-274.0	-198.1	-142.7	-117.0	-113.4	-114.9	-129.2	-155.0	-178.5	-188.4
12.00	-83.1	-74.2	-56.8	-43.1	-36.5	-35.6	-35.2	-40.5	-47.5	-54.0	-56.9
15.00	-17.8	-16.5	-13.5	-10.7	-9.5	-9.3	-9.5	-9.5	-10.8	-12.0	-12.6
	0°	45°	50°	55°	60°	90°	95°	100°	105°	180°	
7.00							-565.9	-564.2	-560.8		
7.10						-569.8	-568.2	-567.7	-567.4		
7.20						-564.0	-562.8	-563.4	-565.4		
7.30				-584.5	-599.3						
7.40			-575.3	-598.0	-600.5						
7.50			-593.5	-600.8							
7.60		-586.4	-599.6	-595.7							
7.70			-596.8								
7.90										-490.3	
8.10										-498.5	
8.20										-492.5	
8.40	-662.1										
8.60	-664.2										

TABLE A.2 – Paramètres donnés par R.R. Toczyłowski [4] pour Ar-HCN

k	0	1	2	3	4	5
b_k	1.48547	0.05926	-0.07533	0.12892	0.04978	-0.00004
c_k	-134348.77	-281277.47	-57041.638	-116961.41		
d_k	14.23209	0.99248	0.88287	0.70551	0.38213	0.06818
$g_{k,0}$	0.06053	-0.12937	0.07849	-0.05992	0.04541	0.03519
$g_{k,1}$	0.00640	0.04016	-0.02325	0.02468	-0.02338	-0.01536
$g_{k,2}$	-0.00277	-0.00416	0.00221	-0.00239	0.00392	0.00207
$g_{k,3}$	0.00013	0.00015	-0.00006	0.00003	-0.00021	-0.00009

TABLE A.3 – Paramètres calculés retenus pour la surface n°1

k	0	1	2	3	4	5
b_k	1.4834580	0.1119314	-0.0328287	0.0943807	0.0338085	-0.0046286
c_k	135784.75	-325678.75	-57995.352	-125775.77		
d_k	14.271529	1.2096562	1.1275672	0.4664125	0.2638973	0.0436920
$g_{k,0}$	0.0488596	-0.1666354	0.0319881	-0.0324706	0.0148494	0.0101494
$g_{k,1}$	0.0098652	0.0557663	-0.0097596	0.0149440	-0.0099116	-0.0055877
$g_{k,2}$	-0.0032256	-0.0059935	0.0012635	-0.0013006	0.0020280	0.0008308
$g_{k,3}$	0.0001521	0.0002130	-0.0000591	-0.0000091	-0.0001256	-0.0000343

TABLE A.4 – Scores relatifs aux paramètres obtenus pour chaque méthode sur la surface n°1

Paramètres	R²	RMSE (mE_h)	MAE (mE_h)
R.R. Toczyłowski	0.99735712	0.14053713	0.07967955
curve_fit + method='lm'	0.99999981	0.00119432	0.00087006
curve_fit + method='lm' + sigma=V_ref	0.99999102	0.00819351	0.00276581
curve_fit + method='trf'	0.99999976	0.00133452	0.00094147
curve_fit + method='trf' + sigma=V_ref	0.99992474	0.02371524	0.00688135
least_squares	0.99999976	0.00133410	0.00094176
least_squares + x_scale='jac'	0.99999981	0.00119432	0.00087006
least_squares + x_scale='jac' + loss='soft_l1'	0.99999981	0.00119432	0.00087005
least_squares + x_scale='jac' + loss='huber'	0.99999981	0.00119432	0.00087006

TABLE A.5 – Paramètres sélectionnés pour la surface n°2

k	0	1	2	3	4	5
b_k	1.4330263	0.0357134	-0.1607479	-0.0121635	-0.0457344	0.0148357
c_k	-119043.41	-234068.29	-42215.107	-124876.08		
d_k	13.375954	-0.3189892	1.4543580	-0.2674760	-0.0000507	0.0396656
$g_{k,0}$	0.1870131	0.0429435	0.0324089	0.2536370	0.1716549	0.0419323
$g_{k,1}$	-0.0181239	0.0155891	-0.0306494	-0.0901479	-0.0594787	-0.0183153
$g_{k,2}$	-0.0009014	-0.0039747	0.0040008	0.0100961	0.0069931	0.0024678
$g_{k,3}$	0.0000112	0.0001696	-0.0000549	-0.0003445	-0.0002870	-0.0001049

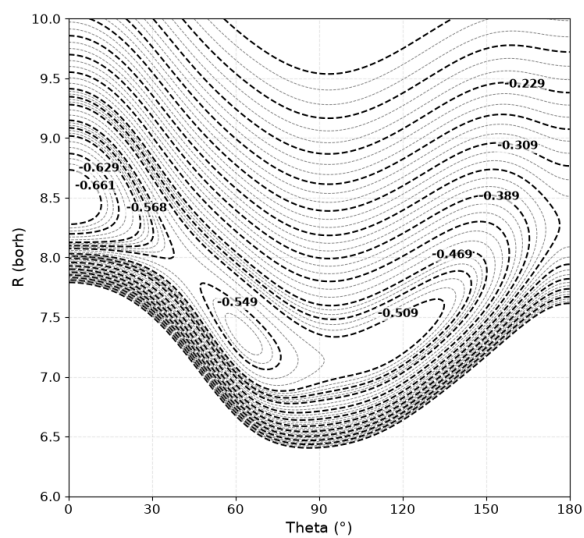
TABLE A.6 – Paramètres sélectionnés pour la surface n°3

k	0	1	2	3	4	5
b_k	1.3927515	0.0969300	-0.0598408	0.0979666	0.0440331	-0.0066774
c_k	-135783.59	-325692.43	-57995.311	-125784.85		
d_k	14.356223	1.2155114	1.1171694	0.4725050	0.2654722	0.0397397
$g_{k,0}$	0.0421958	-0.0743663	0.0044449	-0.0055691	0.0510967	-0.0110268
$g_{k,1}$	0.0012035	0.0230020	0.0016336	-0.0014390	-0.0208052	0.0056295
$g_{k,2}$	-0.0014415	-0.0031206	-0.0005978	0.0011483	0.0027666	-0.0009803
$g_{k,3}$	0.0000770	0.0001784	0.0000445	-0.0001172	-0.0001208	0.0000578

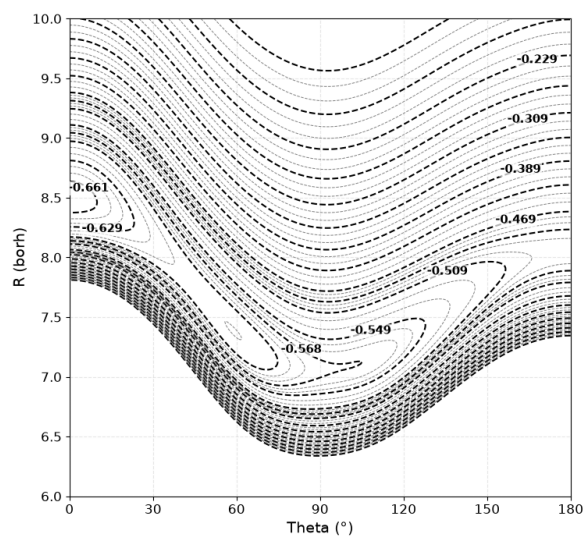
A.2 Figures

FIGURE A.1 – Contour plot de l'énergie potentielle pour Ar-HCN (surface n°1)

(a) Paramètres de R.R. Toczyłowski (A.2)



(b) Paramètres calculés retenus (A.3)



(c) Figure de référence [2]

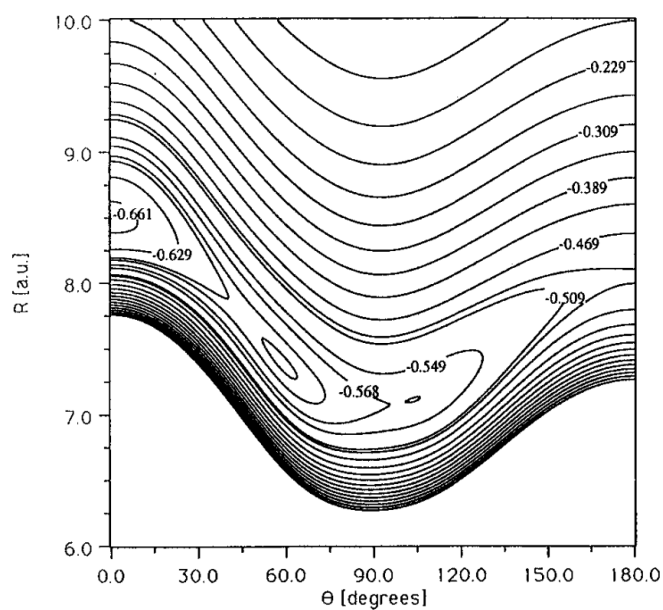


FIGURE A.2 – Coupe de la PES pour $R = 7.5 a_0$ sur quatre configurations de `least_squares` avec un même résidu défini par l'erreur absolue – surface n°2

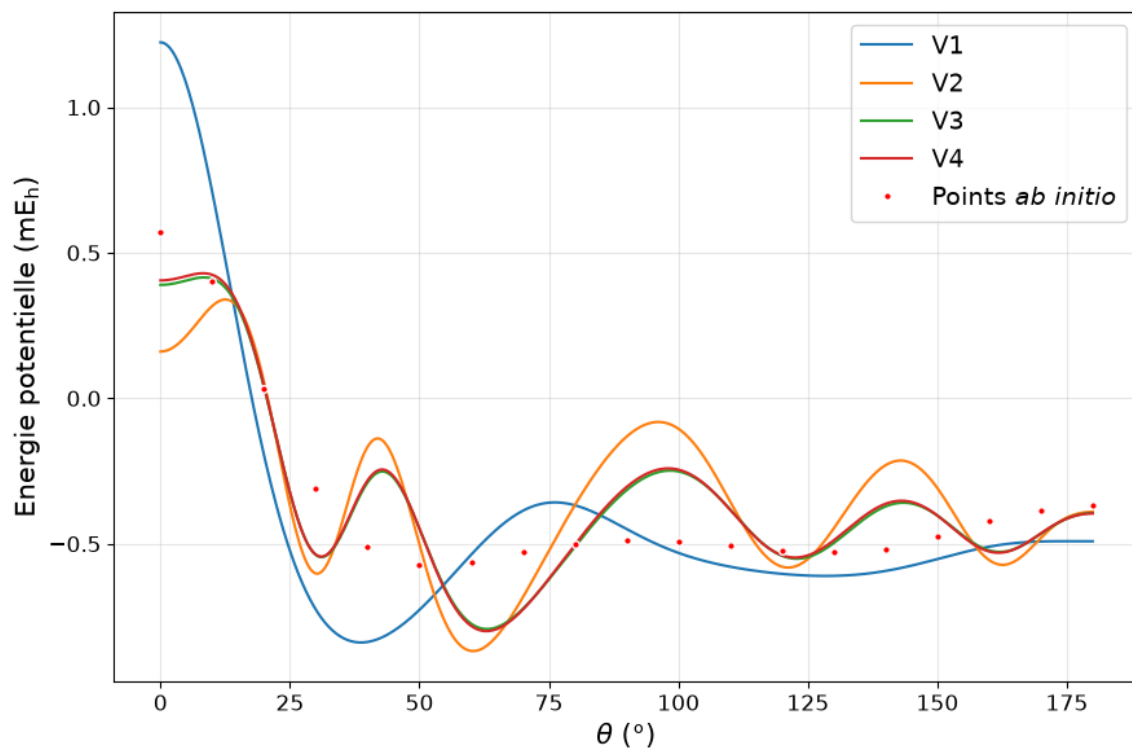


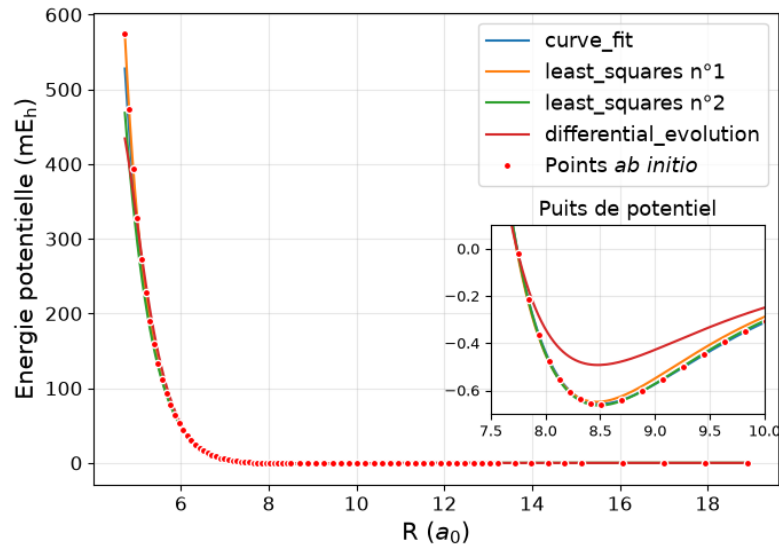
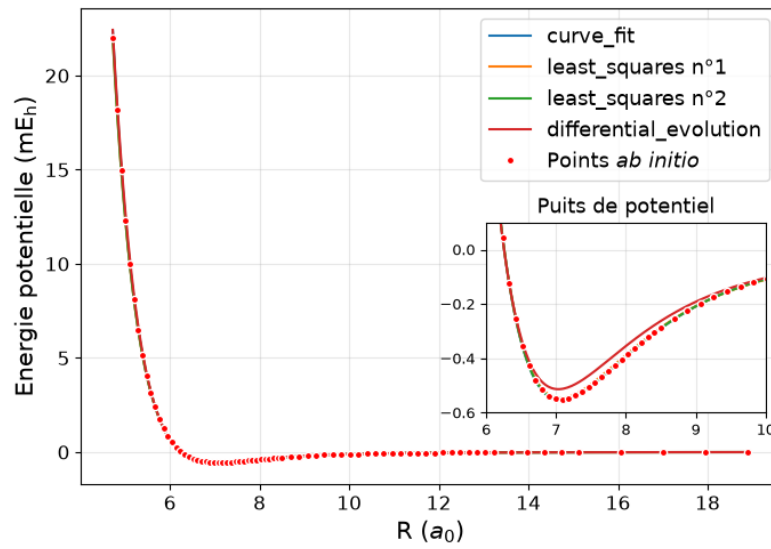
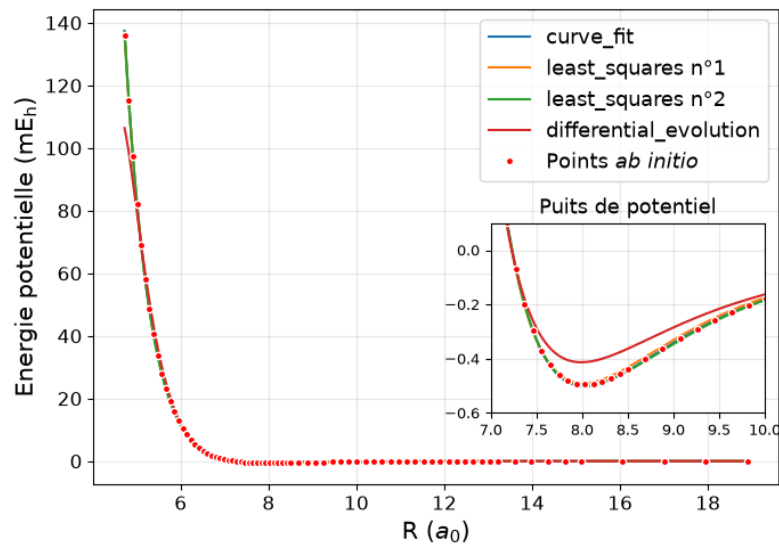
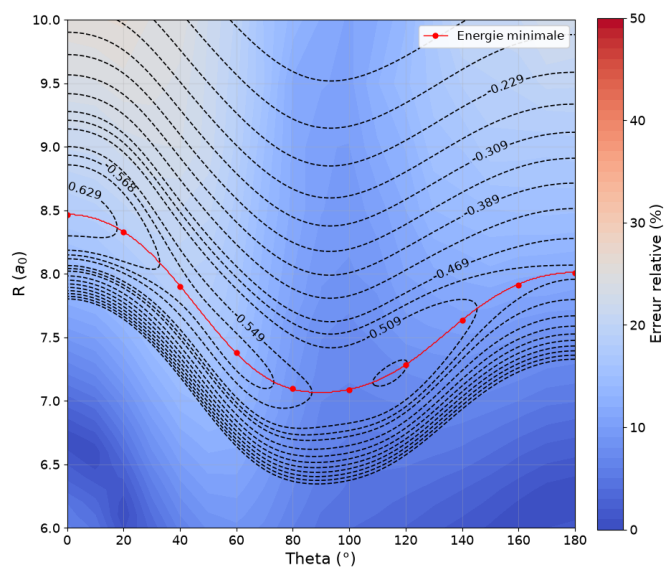
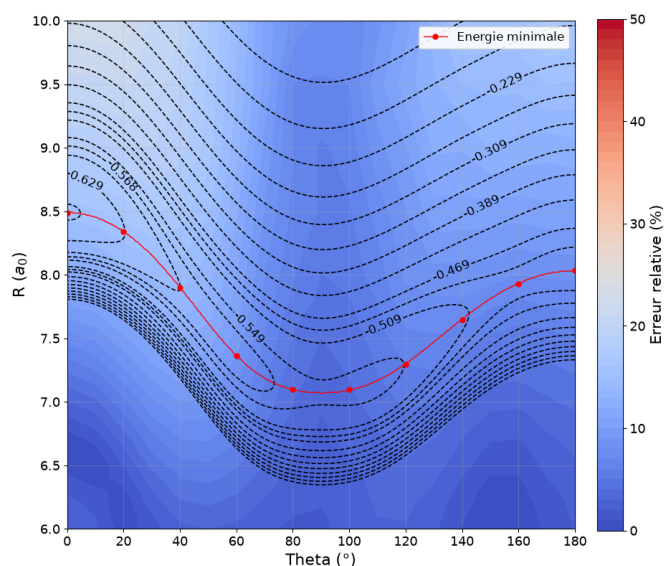
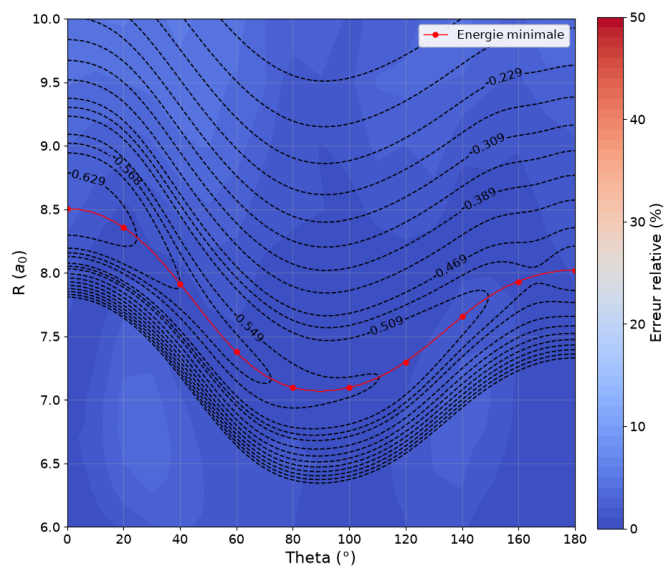
FIGURE A.3 – Coupes de surface pour θ fixé – Ar-HCN (surface n°2)(a) $\theta = 0^\circ$ (b) $\theta = 90^\circ$ (c) $\theta = 180^\circ$ 

FIGURE A.4 – Contour plot de l'énergie potentielle pour Ar-HCN (surface n°2)

(a) Paramètres issus de `least_squares` n°1(b) Paramètres issus de `least_squares` n°2(c) Paramètres issus de `curve_fit`

A.3 Code Source

```

1 def G(R,Theta,g0,g1,g2,g3):
2     g = 0.
3     for i in range(6):
4         g = g + (g0[i]+g1[i]*R+g2[i]*R*R+g3[i]*R*R*R)
5             * eval_legendre(i,math.cos(Theta))
6     return g
7
8 def X(Theta,x):
9     y = 0.
10    for i in range(6):
11        y = y + x[i] * eval_legendre(i,math.cos(Theta))
12    return y
13
14 def f(n,x):
15    y = 0.
16    for k in range(n+1):
17        y = y + (x**k)/factorial(k)
18    y = 1. - math.exp(-x)*y
19    return y
20
21 def V(vars, params):
22
23    R, Theta = vars
24
25    Theta_rad = math.radians(Theta)
26
27    b = params[0:6]      # b0, b1, b2, b3, b4, b5
28    c = params[6:10]     # c0, c1, c2, c3
29    d = params[10:16]    # d0, d1, d2, d3, d4, d5
30    g0 = params[16:22]   # g00, g01, g02, g03, g04, g05
31    g1 = params[22:28]   # g10, g11, g12, g13, g14, g15
32    g2 = params[28:34]   # g20, g21, g22, g23, g24, g25
33    g3 = params[34:40]   # g30, g31, g32, g33, g34, g35
34
35    abs_BR = math.fabs(X(Theta_rad,b)*R)
36    cosTh = math.cos(Theta_rad)
37
38    V_sh = G(R,Theta_rad,g0,g1,g2,g3)
39        * math.exp(X(Theta_rad,d)-X(Theta_rad,b)*R)
40
41    V_as = f(6,abs_BR) * (c[0]*eval_legendre(0,cosTh)\
42                        + c[2]*eval_legendre(2,cosTh))/(R**6)\
43        + f(7,abs_BR) * (c[1]*eval_legendre(1,cosTh)\
44                        + c[3]*eval_legendre(3,cosTh))/(R**7)
45
46    return V_sh + V_as

```

Listing A.1 – Implémentation primaire du modèle analytique de V


```

1 import numpy as np
2 from numpy.polynomial.legendre import legval
3 from scipy.special import eval_legendre
4
5 def f6_opt(x):
6     y = 1.0 + x * (1.0 + x * (0.5 + x * (1.0/6.0 + x * (1.0/24.0 + x *
7         (1.0/120.0 + x / 720.0))))))
8     return 1.0 - np.exp(-x) * y
9
10 def f7_opt(x):
11     y = 1.0 + x * (1.0 + x * (0.5 + x * (1.0/6.0 + x * (1.0/24.0 + x *
12         (1.0/120.0 + x * (1.0/720.0 + x / 5040.0))))))
13     return 1.0 - np.exp(-x) * y
14
15 def V_opt(p, R, Theta):
16     R = np.asarray(R, dtype=float)
17     cosTh = np.cos(np.deg2rad(np.asarray(Theta, dtype=float)))
18
19     b = p[0:6]
20     c = p[6:10]
21     d = p[10:16]
22     g0 = np.asarray(p[16:22], dtype=float)
23     g1 = np.asarray(p[22:28], dtype=float)
24     g2 = np.asarray(p[28:34], dtype=float)
25     g3 = np.asarray(p[34:40], dtype=float)
26
27     X_b = legval(cosTh, b)
28     X_d = legval(cosTh, d)
29
30     X_bR = X_b * R
31     abs_BR = np.abs(X_bR)
32
33     Pl = eval_legendre(np.arange(6)[: , None], cosTh)
34
35     g = g0[: , None] + R * (g1[: , None] + R * (g2[: , None] + R * g3[: , None]))
36
37     exp_val = np.clip(X_d - X_bR, -200, 200)
38     V_sh = np.sum(g * Pl, axis=0) * np.exp(exp_val)
39
40     inv_R = 1.0/R
41     inv_R6 = inv_R * inv_R * inv_R * inv_R * inv_R * inv_R
42     inv_R7 = inv_R6 * inv_R
43
44     V_as = (f6_opt(abs_BR) * legval(cosTh, [c[0], 0, c[2]])) * inv_R6
45         + f7_opt(abs_BR) * legval(cosTh, [0, c[1], 0, c[3]]) * inv_R7
46
47     V_total = V_sh + V_as
48     V_total = np.nan_to_num(V_total, nan=1e100, posinf=1e100, neginf=-1e100)
49     V_total = np.clip(V_total, -1e100, 1e100)
50
51     return V_total

```

Listing A.2 – Implémentation optimisée du modèle analytique de V

```
1 def relative_error_loss(predictions_norm, targets_norm,\  
2     E_mean, E_std, epsilon=1.0):  
3     predictions_raw = (predictions_norm * E_std) + E_mean  
4     targets_raw = (targets_norm * E_std) + E_mean  
5  
6     erreur_absolue = torch.abs(predictions_raw - targets_raw)  
7  
8     erreur_relative = erreur_absolue / (torch.abs(targets_raw) + epsilon)  
9  
10    return torch.mean(erreur_relative)
```

Listing A.3 – Fonction de loss alternative pour le réseau de neurones (Erreur relative)